BinarySpartan: Spartan over binary fields

Srinath Setty

Microsoft Research

Abstract.

Spartan is a SNARK for R1CS that can be instantiated with any multilinear polynomial commitment scheme. We instantiate Spartan over a binary field, using Ligerito as the commitment scheme along with the ring-switching technique of Diamond and Posen; we refer to the instantiation as BinarySpartan. It is transparent, so it requires no trusted setup, and it provides polylogarithmic-sized proofs. Its security rests on a hash function, so it is plausibly post-quantum secure. We apply well-known optimizations to Spartan and sum-check.

We implement and evaluate BinarySpartan end to end. On a MacBook Pro M4 Max (using only its 12 performance cores, and without GPU/Metal acceleration), BinarySpartan proves BLAKE3 at 872,000 hashes/second, SHA-256 at 401,000 hashes/second, and Keccak-f at 287,000 permutations/second, including witness generation. All three clear the 200,000 hashes/second rate proposed as sufficient for a post-quantum Ethereum transition, as well as the roughly 37,000–180,000 hashes/second a possible post-quantum Bitcoin transition would require.

At comparable security, BinarySpartan is 16.3–37.8× faster than Plonky3, 6.3–15.7× faster than Binius64, and 4.0× faster than Hashcaster, which only supports Keccak. On all three hash functions, BinarySpartan's peak throughput matches or exceeds that of Flock, the prior state-of-the-art system. On BLAKE3 and Keccak, BinarySpartan also reaches those rates on far less memory: Flock needs 1.9× and 5.3× more to reach its own best rates. We also evaluate BinarySpartan on the Ethereum Foundation's client-side proving benchmark, where it proves a single SHA-256 of a 2 KiB message in 6.2 ms, the fastest in the suite and 4.2× faster than Flock.

1 Introduction

Spartan [37] is a SNARK for R1CS that can be instantiated with any multilinear polynomial commitment scheme. In a nutshell, Spartan's prover commits to a purported R1CS witness using a polynomial commitment scheme. It then runs two invocations of the sum-check protocol [28] in sequence, the outer and the inner sum-check, which together reduce R1CS satisfiability to an evaluation of the witness polynomial and evaluations of the sparse polynomials that encode the R1CS matrices. The witness evaluation is resolved by the commitment scheme's evaluation argument. The verifier either computes the sparse evaluations on its own or the prover proves them using Spark, Spartan's sparse polynomial commitment scheme.[1]

[1] Spark commits to the matrix polynomials once, in a preprocessing step; only their evaluations are proven online. When the matrices are small enough for the verifier to evaluate on its own, no such preprocessing is needed and the witness is the only committed polynomial.

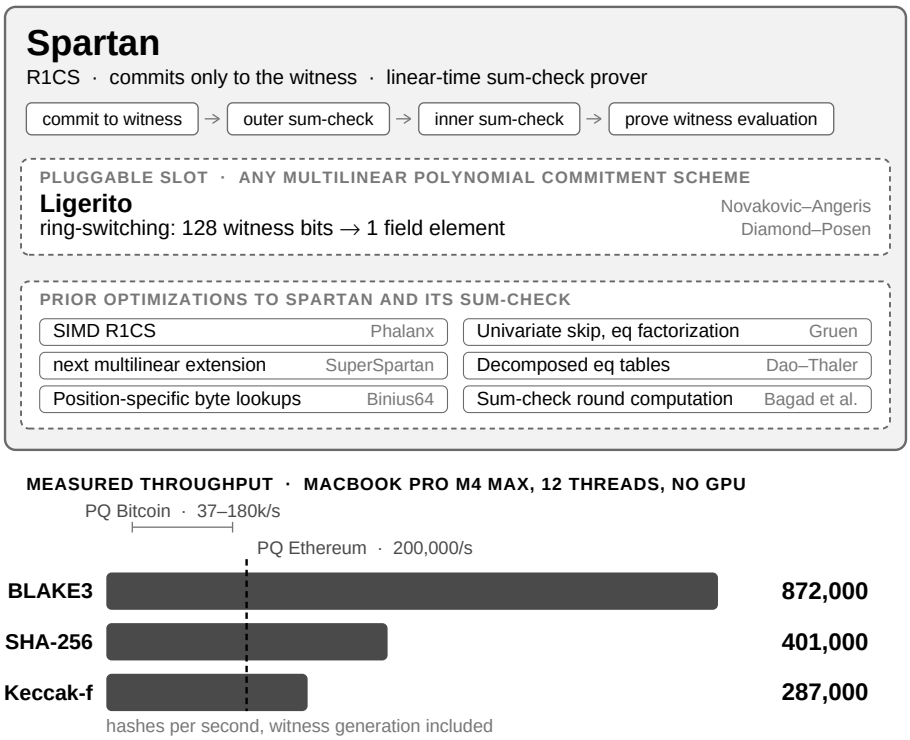


Fig. 1. BinarySpartan at a glance: Spartan [37] over a binary field, instantiated with the Ligerito [32] polynomial commitment scheme and ring-switching [12], along with well-known optimizations to Spartan and to its underlying sum-check [41,39,14,10,2,19,20]. The row below the header is the Spartan protocol. Throughputs are the 12-thread measurements of Table 1 and include witness generation. The 200,000/s line is the rate proposed as sufficient for a post-quantum Ethereum transition [8]; the post-quantum Bitcoin range is derived in Appendix A.

Spartan has been instantiated over prime fields with group-based commitments [37,38,24,1,22], with hash-based commitments [13,42,35], and with lattice-based commitments [25,21], and over the integers and the rationals [6,9,11]. Its generalization to CCS, SuperSpartan [39], has also been instantiated with hash-based commitments [27]. An early-stopping version of Spartan yields a folding scheme [23], which has lattice-based instantiations [4,5,30,31].

Over binary fields, prior works [7,18] raise efficiency concerns about instantiating Spartan. Flock [7, Remark 3] observes that a straightforward implementation of Spartan over $\mathbb{F}_2$ is inefficient and remarks that "this is why the Spartan protocol is designed for R1CS over large fields". Binius64 [18] raises a separate concern, about Spartan's inner sum-check and sparse polynomial evaluation argument, and adopts a different arithmetization instead. Both concerns were addressed by

techniques that predate both Flock and Binius64 and are not specific to binary fields: Gruen's univariate skip [14] for the first, and Phalanx's SIMD R1CS batch structure [41] for the second. We return to them in Section 2.

A concrete efficiency target is proving 200,000 hashes/second, the rate proposed as sufficient for a post-quantum Ethereum transition [8]. (A possible post-quantum Bitcoin transition, where a block's transaction signatures are replaced by a single proof of their validity, implies a target of the same order, roughly 37,000–180,000 compressions/second; see Appendix A.) We ask: can a binary-field instantiation of Spartan, with only widely-used techniques, reach that proving throughput on standard hash functions (e.g., SHA-256, BLAKE3)? We find that it can: Spartan over binary fields proves BLAKE3 at 872,000 hashes/second, SHA-256 at 401,000 hashes/second, and Keccak-f at 287,000 permutations/second—4.4×, 2.0×, and 1.4× the target rate.

To this end, we instantiate Spartan with Ligerito [32] as the multilinear commitment scheme along with the ring-switching technique of Diamond and Posen [12] to efficiently commit to binary witnesses. The resulting instantiation, which we call BinarySpartan, is transparent, so it requires no trusted setup, and it provides polylogarithmic-sized proofs. Its security rests on a hash function, so it is plausibly post-quantum secure. We apply several now-standard optimizations to Spartan and its underlying sum-check protocol:

- the SIMD R1CS of Phalanx [41] to efficiently prove many R1CS instances defined over the same R1CS matrices;
- the next multilinear extension of SuperSpartan [39] to check input and output consistency across neighboring SIMD instances;
- Gruen's univariate skip and factorization of the equality polynomial eq [14];
- optimizations to the sum-check round work: the sublinear-sized decomposed eq tables of Dao and Thaler [10] and the round-computation improvements of Bagad, Dao, Domb, and Thaler [2]; and
- position-specific byte lookup tables that speed up the early rounds of Spartan's outer sum-check, following Binius64 [19,20]: for the bit-valued linear map there, at each byte position we precompute the 256 subset sums of its basis images and XOR one selected entry per byte.

All but the last are not specific to binary fields, and are widely used in recent Spartan implementations over prime fields [35,21]. Thus, BinarySpartan is a natural binary-field instantiation of modern Spartan: Spartan with these well-known optimizations incorporated. We implement and evaluate it end to end. Figure 1 summarizes these ingredients and the throughput they achieve.

Performance. On a MacBook Pro M4 Max, using only its 12 performance cores and with no GPU acceleration, BinarySpartan proves BLAKE3 at 872,000 hashes/second, SHA-256 at 401,000 hashes/second, and Keccak-f at 287,000 permutations/second, including witness generation.

BinarySpartan's throughput is 16.3–37.8× higher than Plonky3 [33], 6.3–15.7× higher than Binius64 [17], and 4.0× higher than Hashcaster [40]. Of these, only Binius64 implements all three hash functions: Plonky3 implements BLAKE3 and Keccak, and Hashcaster implements Keccak.

BinarySpartan outperforms Flock [7] by 8.6% on BLAKE3, and matches it on SHA-256 and Keccak-f, where the two are within measurement error of each other (Section 4.1). Flock is likewise Spartan over binary fields with well-known optimizations, plus further optimizations of its own; BinarySpartan includes only optimizations that pre-date Flock (Section 3).

BinarySpartan is also fast on a single hash, without giving up any of that throughput. On the Ethereum Foundation's client-side proving benchmark, it proves one SHA-256 of a 2 KiB message in 6.2 ms and of a 128 B message in 3.5 ms. On the same benchmark, it is 4.2–7.0× faster than Flock, the next fastest scheme, and at least 6.4× faster than Binius64 [17] and Vega [22] (Section 4.3).

GPU/Metal proving. All performance reported in this paper is CPU-only: no prover work is offloaded to a GPU (e.g., Metal on Apple silicon). Doing so can produce a significant increase in throughput on top of what we report. For instance, an optimization contest has produced a port of Flock's BLAKE3 prover to Metal on Apple silicon [26], which achieves a speedup over the CPU prover. We expect similar speedups in BinarySpartan when offloading work to the GPU.

2 Spartan's efficiency over binary fields

Prior work raises two efficiency concerns about Spartan for R1CS over $\mathbb{F}_2$: Flock points at Spartan's outer sum-check and notes that a straightforward prover implementation must perform excessive extension field operations [7, Remark 3]. Binius64 [18] points at Spartan's inner sum-check and sparse polynomial evaluation argument, and notes that with one matrix entry per witness bit, the prover pays at least $O(\lambda)$ bit operations per bit, where λ is the security parameter.

Both concerns were addressed before both Flock and Binius64, by techniques that are not specific to binary fields: Spartan implementations over large prime fields use them too [21].

The outer sum-check concern is answered by the univariate skip and the sublinear-sized `eq` tables. Soundness requires challenges from a large extension field, but it does not force the prover to spend an extension field multiplication per witness bit. Gruen's univariate skip [14] binds the first rounds as a single univariate message whose construction weights `eq` by bits, a selection rather than a multiplication, so extension arithmetic begins only on a domain smaller by the skip factor; and the sublinear-sized `eq` tables [10] avoid materializing the linear-sized one.

The inner sum-check and sparse polynomial evaluation concern is answered by the batch structure. Phalanx's SIMD R1CS [41] shares one set of matrices across every instance in a batch, so the entry count scales with the block and not the batch. Sparse polynomial evaluation proofs do not arise at all: the shared matrices are small enough for the verifier to evaluate directly, so BinarySpartan never invokes Spark, Spartan's sparse polynomial commitment. Phalanx makes the same observation: evaluating the matrices locally costs time linear in the block and independent of the batch size. Even if a large enough block made Spark worthwhile, it would apply only to the shared matrices, and so would scale with the block and not the batch.

3 Related work

Flock. Flock [7] is the closest related work.

- BinarySpartan is Spartan over binary fields with well-known optimizations.
- Flock is Spartan over binary fields with well-known optimizations and further optimizations of its own.

BinarySpartan includes only optimizations that pre-date Flock. Several of the well-known optimizations are shared: BinarySpartan and Flock both use Phalanx's SIMD R1CS, and both commit with Ligerito [32] and the ring-switching protocol of Diamond and Posen [12]. Both target 100 bits of security, but configure Ligerito differently: BinarySpartan stays in the unique-decoding regime and runs no proof of work, whereas Flock extends Ligerito to the list-decoding regime, taking distance and proximity gaps up to the Johnson bound with proximity-gap grinding.

On throughput, BinarySpartan is ahead of Flock by 8.6% on BLAKE3. On SHA-256 the two are level, and on Keccak-f Flock is ahead by 0.3%; both of these differences fall inside the run-to-run variation (Section 4.1). BinarySpartan reaches this from Spartan over binary fields with only the well-known optimizations, and in the more conservative Ligerito configuration. Flock's additional optimizations can be added to BinarySpartan without redesigning it, and since BinarySpartan already matches or exceeds Flock without those optimizations, adopting them should widen the margin. We have not implemented them, and their benefit may partly overlap with optimizations BinarySpartan already applies.

On a single hash, BinarySpartan is 4.2–7.0× faster than Flock (Section 4.3).

Antecedents. The SIMD R1CS formulation and the next extension that BinarySpartan uses both appear earlier in works on delegating computation. Ben-Sasson et al. [3] give a similar R1CS structure for a univariate polynomial IOP. Holmgren and Rothblum [15] give the same next extension, together with an efficient algorithm to evaluate it, where it links adjacent steps of a computation transcript.

4 Implementation and evaluation

We implement BinarySpartan as a single Rust crate. Arithmetic is in $\mathbb{F}_{2^{128}}$ modulo $x^{128} + x^7 + x^2 + x + 1$. The witness is a vector of bits: ring-switching packs $2^7 = 128$ of them into one field element, so committing to 128 witness bits is equivalent to committing to a single field element. We apply Gruen's univariate skip over the first 4 rounds of Spartan's outer sum-check.

Security. We target 100 bits of security. We configure Ligerito to recurse until at most 8 variables remain, then send the terminal polynomial in the clear. Our soundness accounting assumes only the unique-decoding bound, not the wider Johnson radius, and the query count alone provides the 100 bits: we run no proof of work, so no grinding is counted. Summing the query-consistency and proximity-gap terms over every Ligerito level leaves a soundness error of at most $2^{-102.6}$ at the sizes evaluated here, and the bound stays below 2^{-100} up to 2^{28} committed coefficients.

Circuits. We write each hash function as a SIMD R1CS circuit over bits: SHA-256 in 30,376 constraints per compression, BLAKE3 in 14,000, and Keccak-f in 38,400 per permutation. Following Hashcaster [40], our Keccak encoder packs three permutations into one instance. A single permutation fills only 61% of its padded witness block and three fill 92%. Flock [7] packs three as well, so the Keccak comparison is like-for-like.

The single-hash latency experiments use SHA-256, with one compression per instance. SuperSpartan's `next` multilinear extension [39] links each instance's output to the next instance's input, and instance 0's input is pinned. Nothing reads the last instance's output, so we add a tail matrix: linear rows that pin chosen bits of a chosen instance to public values. These rows bind the final digest as a public output and check the padding bytes against the claimed message length. The last instance's index depends on the message length, so it is public and travels with the instance rather than the shape, and one circuit serves every message length. The rows pin variables that already exist, so they take SHA-256 to 31,656 constraints and commit nothing further.

Baselines. We follow Flock's experimental setup. We compare against four provers: Flock [7], Binius64 [17], Plonky3 [33], and Hashcaster [40]. The baselines, their pinned commits, and their configurations are Flock's, and we run all of them through Flock's own benchmark harness at `8790722`, which builds each baseline from its pinned commit and runs it, and which is also where we measure Flock itself. That harness pins Binius64 at `8f21b348`, Plonky3 at `109e95c1`, and Hashcaster through the `bench-hash-in-snark` harness at `1af6fc55`; it also patches Binius64 to generate its witness in parallel, since its stock witness generator is single-threaded, a change in Binius64's favor. We set the batch grids, size caps, thread count, and repetition counts through the harness's environment variables, adding the last of these to Flock's benchmark drivers as variables that default to their upstream values. Beyond Flock's own Binius64 patch, no prover code is modified.

The baselines do not all target 100 bits. Plonky3's default configuration targets 113 bits under proximity-gap conjectures and proves 65; Flock reconfigures it to 206 queries with 16-bit query grinding for roughly 100 proven bits, and we measure that reconfigured Plonky3, which is slower than the default. Our margin over Plonky3 is therefore a margin at matched security, not one over Plonky3's fastest configuration. Binius64 targets 96 bits and Hashcaster 100, against our 100, so our margin over Binius64 is understated.

Hardware. All measurements are on a MacBook Pro M4 Max with 12 performance cores, 4 efficiency cores, and 128 GB of memory, without GPU acceleration, on an otherwise idle machine. We run every prover on the 12 performance cores and leave the efficiency cores idle, which is where Flock is designed to run [36]; BinarySpartan is not tuned for heterogeneous scheduling either. Enabling the efficiency cores raises BinarySpartan's throughput and lowers Flock's and Binius64's, so the configuration we report is the one less favorable to BinarySpartan.

Prover	BLAKE3	SHA-256	Keccak-f
Plonky3	53,634	—	7,604
Binius64	55,488	45,474	45,786
Hashcaster	—	—	72,535
Flock	802,899	400,928	288,315
BinarySpartan	871,796	400,925	287,464

Table 1. End-to-end throughput in hashes/second at 12 threads, including witness generation. A hash here is one SHA-256 or BLAKE3 compression function or one Keccak-f permutation. The Keccak column packs three permutations per instance, except for Binius64 and Plonky3, which do not pack.

Measurement method. We sweep every prover over a range of batch sizes and let each use its own encoder. Following Flock, we sweep until peak memory usage exceeds roughly 10 GB and report each prover's best point below that ceiling; Flock applies that ceiling to every prover it measures, and so do we.

Every point is two warmup repetitions followed by the best of three timed ones. We set that policy ourselves rather than inherit each prover's own, so that it is identical across provers. We repeat the whole sweep in five rounds, each a fresh process, with prover and batch size interleaved in a fresh random order: a sweep run in ascending size order systematically under-measures whatever it leaves for last. We report the mean over the five rounds rather than the fastest. Across rounds at their reported peaks, BinarySpartan's relative standard deviations are 0.8% for SHA-256, 0.7% for BLAKE3, and 0.9% for Keccak, and Flock's are 1.0%, 1.0%, and 1.2%.

The remaining three baselines trail by 4–38×, so their exact figures cannot change a conclusion. They run in two passes ahead of the rounds, so their allocations cannot perturb a randomized point, and are reduced by the same mean. Reported times include witness generation. BinarySpartan's proof sizes are uncompressed `bincode` serializations.

4.1 Peak throughput

Flock, Binius64, and BinarySpartan prove all three hash functions; Plonky3 covers BLAKE3 and Keccak, and Hashcaster only Keccak.

Table 1 gives the results. Against Plonky3, BinarySpartan is 16.3× faster on BLAKE3 and 37.8× faster on Keccak. Against Binius64, it is 15.7× faster on BLAKE3, 8.8× on SHA-256, and 6.3× on Keccak. Against Hashcaster, which only supports Keccak, BinarySpartan is 4.0× faster.

BinarySpartan is ahead of Flock on BLAKE3 by 8.6%. On SHA-256 the two are level, and on Keccak Flock is ahead by 0.3%; both of these differences fall inside the 0.7–1.2% round-to-round variation reported above. BinarySpartan reaches these figures from the more conservative of the two configurations: as Section 3 discusses, it runs Ligerito in the unique-decoding regime with no proof of work, so it answers more queries than Flock does for the same 100 bits of security.

These peaks are attained at the following batch sizes. On BLAKE3, BinarySpartan is at 2^{17}, Flock at 2^{18}, Binius64 at 2^{15}, and Plonky3 at 2^{14}; on SHA-256, BinarySpartan and Flock are at 2^{17} and Binius64 at 2^{16}. Keccak batches are not all powers of two, because the provers pack different numbers of permutations per instance, so we give them as permutations: BinarySpartan at 12,288, Flock at 98,304, Binius64 at 16,384, Plonky3 at 5,461, and Hashcaster at 393,216.

BinarySpartan proves BLAKE3 at 872,000 hashes/second, SHA-256 at 401,000 hashes/second, and Keccak-f at 287,000 permutations/second, clearing the 200,000 hashes/second target proposed as sufficient for a post-quantum Ethereum transition [8] on all three. It also clears the comparable target implied by a possible post-quantum Bitcoin transition (Appendix A).

Proof size. At a common batch of 2^{14} BLAKE3 hashes, BinarySpartan's proof is 519 KiB and Flock's is 357 KiB. BinarySpartan sends an independent Merkle path for every query instead of deduplicating the nodes those paths share, so it repeats them. Compressing both serialized proofs with `gzip -9` recovers most of that redundancy and narrows the gap to 422 KiB against 355 KiB; we report this as a proxy for what explicit path deduplication would achieve.

Verifier time. Deserializing and verifying a proof takes 2.7–4.1 ms at the configurations of Table 1: 2.7 ms at 2^{17} BLAKE3 hashes, 2.7 ms at 2^{17} SHA-256 hashes, and 4.1 ms at 12,288 Keccak permutations. Verifier time grows slowly in the batch size: increasing the SHA-256 batch eightfold from 2^{13} to 2^{16} costs only 1.28×.

4.2 Throughput with varying batch sizes

Table 1 reports each prover at its own best batch size. We now measure BinarySpartan and Flock across batch sizes instead.

Figure 2 shows the BLAKE3 sweep. At the smallest batch of each hash function BinarySpartan proves 37.5% more BLAKE3 hashes per second than Flock, 19.6% more SHA-256 hashes, and 23.4% more Keccak permutations. All three margins are larger than at the peaks of Table 1. The sweep covers six batch sizes per hash function. BinarySpartan leads at 15 of the 18 points and matches Flock at one, the SHA-256 peak. It trails at the remaining two, by 4.0% and 5.4%. Both are Keccak batches past the 12,288 permutations at which BinarySpartan itself peaks, where BinarySpartan's rate is falling while Flock's is still rising: at its own peak rate BinarySpartan would be 0.2% ahead of Flock at 49,152 and 0.3% behind at 98,304, both inside the round-to-round spread.

On BLAKE3 and Keccak, BinarySpartan reaches its best rates on far less memory than Flock. It peaks at 3.92 GB on BLAKE3 and 1.25 GB on Keccak, where Flock needs 7.42 and 6.67 GB for its own best rates. SHA-256 is the exception: both systems peak at the same batch, and BinarySpartan uses 7.56 GB against Flock's 7.14. BinarySpartan also comes within 10% of its peak rate on far smaller batches—2^{15} BLAKE3 hashes, 2^{14} SHA-256 hashes, and 12,288 Keccak permutations, each in 1.3 GB or less—and needs half the batch Flock does to stay within the same 10%.

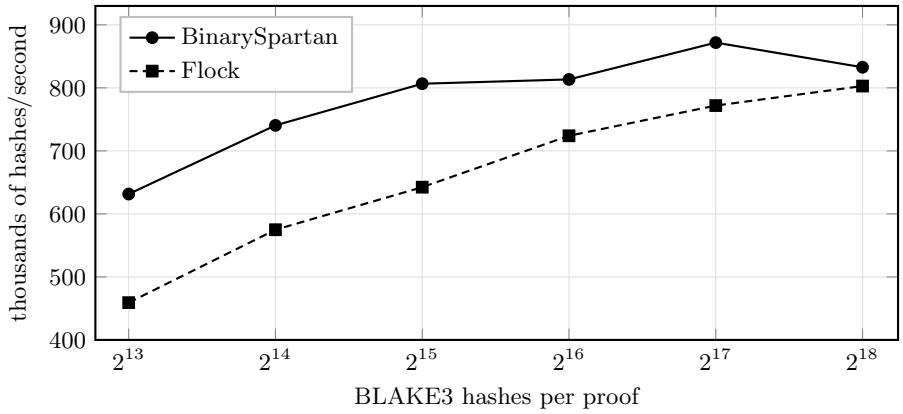


Fig. 2. BLAKE3 throughput against batch size for BinarySpartan and Flock. Each point is the mean over the same five rounds as Table 1, and both provers are measured under the protocol described there. BinarySpartan is ahead at every batch size, and is within 10% of its own peak by 2^{15} hashes, where Flock needs 2^{16}. Both sweeps stop at 2^{18}, the largest batch within the memory ceiling. SHA-256 and Keccak-f have the same shape, with BinarySpartan's lead again largest at the smallest batch.

Every batch so far fits under the 10 GB ceiling. We also measured the baselines at larger batches. On BLAKE3 and SHA-256 the larger batches do not help Flock, which is already past its peak at 7.4 and 7.1 GB: the next size up needs 14.6 and 14.3 GB and is 1.3% and 0.2% slower. Keccak is the one hash function where a larger batch still helps Flock, but only marginally: at 4× the batch, needing 26.6 GB, Flock gains 4.4% and reaches 301,089 permutations/second. That is 4.7% above BinarySpartan, which reaches its own Keccak peak at 1.25 GB, 21× less memory. The ceiling ends BinarySpartan's own SHA-256 sweep the same way, one size short of roughly 15 GB. Among the other three, Hashcaster gains only 2.2% at twice the batch and 11.2 GB and stays nearly 4× behind BinarySpartan, and Plonky3 is 9% below its peak at 10.6 GB.

4.3 Latency

We also measure the time to prove a *single* SHA-256, using the Ethereum Foundation's client-side proving benchmark and its harness [34]. Here we compare against Flock [7], Binius64 [17], and the two configurations of Vega [22], single-circuit (Vega$_{\mathsf{SC}}$) and multi-circuit (Vega$_{\mathsf{MC}}$). For BinarySpartan, we measure all five message sizes in 20 independently shuffled rounds and report the mean. We omit the other systems in the client-side benchmark because their published SHA-256 proving times exceed those in Table 2. Both Vega provers split into a reusable preprocessing phase and an online phase, and we quote the online phase alone. Binius64 and both Vega configurations are zero-knowledge, which BinarySpartan and Flock are not.

Message	Binius64	Vega_{SC}	Vega_{MC}	Flock	BinarySpartan
128 B	49.31	24.65	25.35	24.26	3.47
256 B	49.92	25.14	28.23	24.53	3.88
512 B	53.80	36.70	32.90	25.04	4.22
1 KiB	56.65	59.89	37.98	25.66	4.75
2 KiB	62.02	104.19	44.23	26.01	6.24

Table 2. Time in milliseconds to prove one SHA-256, at 12 threads. BinarySpartan, Flock, and Binius64 include witness generation; the Vega columns are the online phase only. This table isolates hashing. Vega targets statements over signed data, which combine hashing with signature verification and parsing; its native 256-bit field arithmetic is an advantage on that workload that the binary-field provers here lack. This comparison does not reflect it.

Table 2 gives the results. BinarySpartan is fastest at every message size. It is 4.2–7.0× faster than Flock, at least 6.4× faster than either Vega configuration, and 9.9–14.2× faster than Binius64. Flock proves the same relation as BinarySpartan here: it also binds the digest as a public output, checks the padding against the claimed message length, and chains across blocks.

Acknowledgments

We thank Ron Rothblum and Justin Thaler for helpful conversations and comments on a prior version of this paper.

References

1. Arun, A., Setty, S., Thaler, J.: Jolt: SNARKs for VMs via lookups. In: EUROCRYPT (2024)
2. Bagad, S., Dao, Q., Domb, Y., Thaler, J.: Speeding up sum-check proving. Cryptology ePrint Archive, Paper 2025/1117 (2025)
3. Ben-Sasson, E., Chiesa, A., Goldberg, L., Gur, T., Riabzev, M., Spooner, N.: Linear-size constant-query IOPs for delegating computation. In: TCC (2019)
4. Boneh, D., Chen, B.: LatticeFold: A lattice-based folding scheme and its applications to succinct proof systems. Cryptology ePrint Archive, Paper 2024/257 (2024)
5. Boneh, D., Chen, B.: LatticeFold+: Faster, simpler, shorter lattice-based folding for succinct proof systems. Cryptology ePrint Archive, Paper 2025/247 (2025)
6. Bünz, B., Fisch, B., Szepieniec, A.: Transparent SNARKs from DARK compilers. In: EUROCRYPT (2020)
7. Bünz, B., Rothblum, R.D., Wang, W.: Flock: Fast proving for batch Boolean computations. Cryptology ePrint Archive, Paper 2026/1329 (2026)
8. Buterin, V.: Possible futures of the Ethereum protocol, part 4: The verge. `https://vitalik.eth.limo/general/2024/10/23/futures4.html` (2024)
9. Campanelli, M., Hall-Andersen, M.: Fully succinct arguments over the integers from first principles. Cryptology ePrint Archive (2024)
10. Dao, Q., Thaler, J.: More optimizations to sum-check proving. Cryptology ePrint Archive, Paper 2024/1210 (2024)
11. DeStefano, Z., Golub, N., Huang, Z., Zhang, J., Frank, S., Walfish, M.: Spain: Succinct proofs for numerical computations. Cryptology ePrint Archive (2026)

12. Diamond, B.E., Posen, J.: Polylogarithmic proofs for multilinears over binary towers. Cryptology ePrint Archive, Paper 2024/504 (2024)
13. Golovnev, A., Lee, J., Setty, S., Thaler, J., Wahby, R.: Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In: CRYPTO (2023)
14. Gruen, A.: Some improvements for the PIOP for ZeroCheck. Cryptology ePrint Archive, Paper 2024/108 (2024)
15. Holmgren, J., Rothblum, R.D.: Delegating computations with (almost) minimal time and space overhead. In: FOCS (2018)
16. Hülsing, A., Butin, D., Gazdag, S.L., Rijneveld, J., Mohaisen, A.: XMSS: eXtended Merkle Signature Scheme. IRTF RFC 8391 (2018), https://www.rfc-editor.org/rfc/rfc8391
17. Irreducible Team: Binius64. https://github.com/binius-zk/binius64 (2025)
18. Irreducible Team: Binius64 blueprint: Overview. https://www.binius.xyz/blueprint/overview/ (2025)
19. Irreducible Team: Module for folding words with four russians. https://github.com/binius-zk/binius64/commit/bd41ba7a916ed403e5fa2e5aecdb82561c6a2f16 (2025)
20. Irreducible Team: NTT lookup for univariate AND reduction. https://github.com/binius-zk/binius64/commit/9aa33699f3a0c126810df55fcfbd8ef6a5b2d7d0 (2025)
21. Jolt Team: Jolt codebase. https://github.com/a16z/jolt (2026)
22. Kaviani, D., Setty, S.: Vega: Low-latency zero-knowledge proofs over existing credentials. In: S&P (2026)
23. Kothapalli, A., Setty, S.: HyperNova: recursive arguments for customizable constraint systems. In: CRYPTO. pp. 345–379 (2024)
24. Kothapalli, A., Setty, S., Tzialla, I.: Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In: CRYPTO (2022)
25. LayerZero Labs: Akita. https://github.com/LayerZero-Labs/akita (2026)
26. Layr Labs: Flock BLAKE3 benchmark challenge. https://github.com/Layr-Labs/flock-challenge (2026)
27. Lean Ethereum: leanVM. https://github.com/leanEthereum/leanVM (2026)
28. Lund, C., Fortnow, L., Karloff, H., Nisan, N.: Algebraic methods for interactive proof systems. In: FOCS (Oct 1990)
29. National Institute of Standards and Technology: Stateless hash-based digital signature standard. FIPS 205 (2024), https://doi.org/10.6028/NIST.FIPS.205
30. Nguyen, W., Setty, S.: Neo: Lattice-based folding scheme for CCS over small fields and pay-per-bit commitments. Cryptology ePrint Archive, Paper 2025/294 (2025)
31. Nguyen, W., Setty, S.: Neo and SuperNeo: Post-quantum folding with pay-per-bit costs over small fields. Cryptology ePrint Archive, Paper 2026/242 (2026)
32. Novakovic, A., Angeris, G.: Ligerito: A small and concretely fast polynomial commitment scheme. Cryptology ePrint Archive, Paper 2025/1187 (2025)
33. Plonky3 Team: Plonky3. https://github.com/Plonky3/Plonky3 (2024)
34. Privacy Stewards of Ethereum: Client-side proving benchmarks. https://github.com/privacy-ethereum/csp-benchmarks (2025)
35. PSE: spartan-whir. https://github.com/privacy-ethereum/spartan-whir (2026)
36. Rothblum, R.D.: Personal communication (2026)
37. Setty, S.: Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In: CRYPTO (2020)
38. Setty, S., Lee, J.: Quarks: Quadruple-efficient transparent zkSNARKs. Cryptology ePrint Archive, Report 2020/1275 (2020)

39. Setty, S., Thaler, J., Wahby, R.: Customizable constraint systems for succinct arguments. Cryptology ePrint Archive (2023)
40. Soukhanov, L.: Hashcaster. `https://github.com/morgana-proofs/hashcaster` (2024)
41. Tzialla, I., Kothapalli, A., Parno, B., Setty, S.: Transparency dictionaries with succinct proofs of correct operation. In: NDSS (2022)
42. World Foundation: ProveKit. `https://github.com/worldfnd/ProveKit` (2025)

A A rough post-quantum Bitcoin target

The 200,000 hashes/second figure of Section 1 is a target for a post-quantum Ethereum transition [8]. Here we give the analogous target for Bitcoin. The estimate is deliberately very rough: it is meant only to fix the order of magnitude, not to be a precise requirement.

Scenario. Suppose transactions carry hash-based (hence post-quantum) signatures. A block embeds everything except the signatures, together with a single proof attesting to knowledge of valid signatures for its transactions. One proof thus replaces every signature in the block.

Estimate. A Bitcoin block holds roughly 2,000–3,500 transactions, each of which would carry a hash-based signature. Bitcoin proposals exist for both stateful and stateless schemes; we take the stateless SLH-DSA (SPHINCS+) standard [29], whose freedom from per-signature state makes it the more robust default. What must be proved is the circuit cost of verification: being data-independent, the circuit pays the worst-case Winternitz chains, so verifying one SLH-DSA-128s signature is about 4×10^3 hash compressions, dominated by the one-time-signature chains of its hypertree (over a FORS few-time signature, plus Merkle paths). A representative block is therefore 8–14 million compressions; we take $\approx$11 million. Blocks arrive about every ten minutes (600 s); leaving margin for propagation and reorganizations, a prover targets 10%–50% of the interval, i.e., 60–300 s. The required rate is then $(11 \times 10^6$ compressions$)/(60$–$300\,\text{s}) \approx 37{,}000$–$180{,}000$ compressions/second—the same order as the Ethereum target, and reaching it for a full block at the tightest budget. BinarySpartan clears this range on all three hash functions: SHA-256 at 401,000/s and Keccak at 287,000/s, the families over which SLH-DSA is standardized, and BLAKE3 at 872,000/s (Section 4.1). Stateful XMSS [16] is cheaper, of order 10^3 compressions per signature and 10,000–50,000 compressions/second. The faster SLH-DSA-128f set, or an unusually full block, pushes the aggressive corner past the Ethereum target instead.

Caveats. The estimate omits second-order costs. Our measured throughput is for *independent* compressions, whereas a verification circuit links the compressions within a signature—the chains and Merkle paths—through the `next` extension and does auxiliary address and checksum work, so its effective rate is somewhat lower, which tightens this comparison. Keeping the on-chain proof small may also call for recursion: rather than posting a single monolithic proof over an entire block, the per-batch proofs would be recursively composed into one succinct proof. Chunking a block is cheap in throughput terms—it is some 80 proofs at BinarySpartan's best SHA-256 batch and around 670 at the smallest batch still within 10% of that rate (Section 4.2)—but composition is not: each step proves BinarySpartan's own verifier, which checks SHA-256 Merkle paths and does binary-field work per query. We do not measure that circuit here. Finally, the proof is computed concurrently with mining: a block's signatures are known once its transactions are fixed, so proving runs alongside the proof-of-work search and finishes well within the block interval.